PortPilot - scanner TCP de laboratoire

Durée : 5 heures · Niveau débutant · 3 merge requests

Tu vas écrire un petit programme Python qui essaie une connexion TCP vers quelques ports de 127.0.0.1, conserve un état par port et affiche un bilan. Il n'envoie aucune donnée applicative et ne récupère aucune bannière. Un port ouvert signifie uniquement que la connexion TCP a réussi : cela ne prouve ni une vulnérabilité ni la nature du service.

Le parcours complet et les définitions détaillées des fonctions et des huit tests sont dans le PDF PortPilot. Ce fichier reprend aussi les étapes GitLab et les critères de livraison.

Préparation

Utilise le dépôt autorisé par ton école, son URL et sa branche principale (main ou master). Il te faut Python 3, Git, un runner GitLab actif utilisable par les branches de MR et les droits nécessaires aux règles de fusion. Sans runner ou accès aux règles, tu peux développer et exécuter les tests localement, mais la CI bloquante reste non validée.

Le dossier de cours dans Cyber Atelier est vide. Clone le dépôt de l'école directement dans ce dossier avant de créer tes fichiers, avec git clone <URL_DU_DEPOT> . (le point signifie « ici »). Ainsi, le dépôt ne se retrouve pas dans un sous-dossier. Le terminal web lance chaque commande dans le dossier courant explicite et ne garde pas un cd d'une commande à la suivante. Pour une session Git interactive, ouvre un terminal de ta machine et lance docker compose exec atelier bash depuis le dossier de Cyber Atelier. Initialise le dépôt avec README.md et .gitignore ; ce dernier exclut les caches Python et portpilot.zip. Protège la branche principale selon les règles de l'école : pas de push direct, force push désactivé, fusion après pipeline réussi et option pipeline ignoré comme réussi désactivée. Vérifie que le runner accepte les jobs de branches non protégées et les tags demandés.

Contrats Python

Écris cinq fonctions dans portpilot.py et les tests dans verification.py.

- valider_ports(ports) accepte une liste non vide de 1 à 10 ports distincts, entiers de 1 à 65535 inclus. Elle retourne les mêmes valeurs dans le même ordre. Elle lève ValueError pour une liste vide, plus de dix éléments, un doublon, une valeur hors bornes, un texte, un nombre décimal ou un booléen. Ne convertis pas le texte en entier. Une entrée qui n'est pas une liste est hors contrat.
- tester_port(port) utilise uniquement l'hôte constant 127.0.0.1 et le délai fixe de 0,5 seconde. Elle retourne exactement OUVERT si la connexion réussit, REFUSE pour ConnectionRefusedError, TIMEOUT pour un dépassement du délai et ERREUR pour un autre OSError. Toute socket créée est fermée même en cas d'échec.
- scanner_ports(ports) valide toute la liste avant toute connexion, fait une tentative par port dans l'ordre reçu et retourne une liste de paires [port, état] dans ce même ordre.
- resumer(resultats) retourne cinq compteurs entiers : total, ouverts, refuses, timeouts, erreurs. Les catégories totalisent total. Une liste vide donne cinq zéros.
- main coordonne la liste configurée, le scan, l'affichage d'un résultat par port et le bilan. Pour une liste invalide, affiche un message clair. L'import de portpilot par verification.py ne doit pas lancer main.

Reste dans le périmètre : pas de saisie interactive, d'arguments, de classes, de threads, de paquets pip, de plage de ports, de scan externe, d'exploitation, de lecture de fichiers ou de rapport écrit sur disque.

Comment travailler un exercice

Chaque mission construit une partie du projet. Commence par « Avant de commencer », puis avance palier par palier. Les durées sont des repères : prends le temps de comprendre chaque résultat.

1. Lis l'objectif et les fichiers concernés. Un fichier indiqué comme « à créer » n'est pas fourni : c'est toi qui l'écris.
2. Les cartes de tests donnent l'entrée et le résultat attendu. Écris les tests correspondants dans le fichier indiqué ; cocher un palier ne les exécute pas. Avec unittest, « Ran 0 tests » signifie qu'aucun test n'a été exécuté : ce n'est pas une validation. Un cas décrit peut contenir plusieurs valeurs ; le nombre de méthodes Python n'est donc pas forcément le nombre d'identifiants.
3. Suis les actions numérotées. Les blocs Python se placent dans l'éditeur ; les commandes se lancent au moment demandé. Relis les blocs Git : des modifications et des actions dans GitHub/GitLab peuvent être nécessaires entre deux commandes.
4. Vérifie le résultat indiqué avant de poursuivre. Un échec attendu (phase rouge, argument refusé ou conflit volontaire) est expliqué dans le palier.
5. Si tu bloques, lis la dernière ligne de l'erreur, compare l'entrée et l'attendu, puis ouvre l'indice. Fais un petit changement et relance.

Le bon endroit pour chaque action

Sur ton Mac, ouvre `workspaces/portpilot` dans VS Code. Dans Cyber Atelier, utilise l'éditeur et la console de **Coder**, avec `cwd` sur `.` (la racine du dépôt). Fais la préparation de **Environnement** avant MR1 ; la mission crée ensuite sa branche.

Le code Python se place dans l'éditeur. Les commandes `docker compose ...` se lancent sur ton Mac depuis `cyberatelier` ; `/app/...` existe dans le conteneur. Le laboratoire interrogé doit tourner dans le même environnement que ton programme.

La console web repart à neuf à chaque exécution : un `cd`, une activation de venv ou `echo $?` ne se transmet pas à la commande suivante. Lance le programme et `echo $?` ensemble quand le palier le demande. Le nombre affiché par echo est celui du programme ; la console peut afficher code 0 pour echo lui-même.

Un venv créé sur le Mac ne fonctionne pas dans le conteneur : utilise `python3` dans ce dernier. Remplace `main` par `master` si nécessaire. Si une branche existe déjà, reprends-la avec `git switch NOM` sans `-c`. Après chaque fusion, actualise la branche principale avant la mission suivante.

Les mots utiles

Contrat. Les règles à respecter : arguments reçus, valeur retournée, erreurs et cas limites. Tu choisis le code qui respecte ces règles.

Test à écrire. Une entrée et un résultat attendu à transformer en vérification Python. Les identifiants S1, T1, etc. désignent des cas à couvrir, pas des tests déjà installés.

Rouge puis vert. Écris la vérification, lance-la et comprends son échec. Écris ensuite le code, relance et garde les vérifications précédentes. Un nouveau cas peut déjà passer : ne casse pas le code pour le rendre rouge.

Fonction pure. Une fonction qui calcule son résultat depuis ses arguments, sans accéder au réseau ou au disque et sans modifier les données reçues.

Exception. Une erreur signalée par Python, par exemple ValueError. « Lever » signale l'erreur ; « attraper » la traite avec except. Si le contrat attend une exception, la voir est une réussite du test.

Fixture. Un fichier de données fictives fourni pour rendre un essai reproductible. Une fixture XML ou JSON remplace la sortie d'un outil réel dans les tests.

Mock. Un faux temporaire qui remplace un appel pendant le test. Il permet de choisir le résultat et de vérifier les arguments sans lancer de réseau ou de processus réel.

CLI. L'interface en ligne de commande : tu lances un programme avec des arguments. --help décrit les options ; argparse les lit et vérifie leur forme.

Code de sortie. Le nombre rendu au terminal quand le programme se termine. 0 signifie réussite ; une valeur non nulle signale un échec ou un usage incorrect selon le contrat.

Commit. Un point enregistré dans l'historique Git. git add choisit les fichiers ; git diff --staged montre le contenu choisi ; git commit l'enregistre.

PR / MR. Une demande de fusion de ta branche vers la principale. GitHub dit pull request (PR), GitLab merge request (MR). L'ouvrir, la relire et la fusionner sont trois actions distinctes.

Pipeline / job. GitLab exécute le pipeline : un ensemble de tâches, appelées jobs. La CI est cette exécution automatique. Un fichier de configuration présent ne prouve pas qu'un job a tourné.

Artefact. Un fichier conservé par un job GitLab après son exécution, par exemple le ZIP de livraison.

Empreinte / manifeste. L'empreinte SHA-256 est calculée à partir des octets d'un fichier. Le manifeste mémorise des noms et leurs empreintes pour comparer plus tard.

Parser. Lire un format structuré pour obtenir des données Python. JSON donne des dictionnaires et listes ; XML donne un arbre de balises et d'attributs.

Package / venv. Un package regroupe des modules Python dans un dossier. Un venv est un environnement Python séparé, que tu ne livres pas dans Git.

Déterministe. Avec les mêmes entrées, le résultat est identique. Pour tester un rapport daté, le test fournit la date au lieu de lire l'heure actuelle.

stderr / stdout. Deux sorties du programme : stdout porte le résultat normal, stderr les messages d'erreur et les traces. Le code de sortie reste un nombre séparé.

Loopback. L'adresse de la machine où tourne le programme. 127.0.0.1 dans Docker désigne le conteneur ; sur le Mac, elle désigne le Mac ; dans la CI, elle désigne le runner.

Les trois MR

Les durées sont des repères compris dans les 300 minutes du parcours.

MR1 - feat/validation (80 min)

Tu prouves aussi qu'une régression rend le pipeline rouge et bloque la fusion. Avance dans l'ordre des paliers et vérifie chaque résultat avant de poursuivre.

Avant de commencer

- Point de départ : Un dépôt GitLab cloné, initialisé avec README.md et .gitignore ; l'accès au runner et aux règles de fusion est vérifié.
- Ce que tu vas produire : portpilot.py et verification.py : valider la liste de ports ; .gitlab-ci.yml : exécuter les tests de MR.
- Pourquoi : Tu prouves aussi qu'une régression rend le pipeline rouge et bloque la fusion.
- Commande des tests depuis la racine : `python3 verification.py`. Les cas ci-dessous sont à écrire par toi.

Palier 1 - Partir d’une branche à jour (10 min)

Créer la branche de la MR depuis une branche principale à jour, sans rien emporter d’inutile.

Où tu travailles : Dépôt

Pour comprendre

- Une branche est une ligne de travail séparée de la branche principale (main, ou master selon ton école). Tu codes dessus sans toucher à la version livrée.

Actions dans l’ordre

1. Ouvre le dossier workspaces/portpilot dans VS Code, puis un terminal (Terminal > New Terminal).
2. Vérifie que ton dépôt est propre, puis reviens sur la branche principale et mets-la à jour avant de créer la tienne.
3. Si ton école utilise master, remplace main par master dans les commandes.

Terminal · Relire l’état du dépôt

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git status
```

Terminal · Après la fusion : actualiser la principale

```
git switch main
```

Terminal · Commande à lancer au moment indiqué

```
git pull --ff-only
```

Terminal · Commande à lancer au moment indiqué

```
git switch -c feat/validation
```

Terminal · Relire l’état du dépôt

```
git status
```

Tu sais que c’est bon quand : git status affiche « On branch feat/validation » et « nothing to commit, working tree clean ».

Indice si tu bloques : Si git pull --ff-only refuse, c’est que ta branche principale locale a des commits qui ne viennent pas du dépôt de l’école : ne force rien, demande de l’aide avant de continuer. Si git status signale des fichiers modifiés, lis-les avec git diff avant de décider quoi faire.

Palier 2 - Premier test rouge : T1 (10 min)

Écrire un test qui appelle valider_ports, le voir échouer, et comprendre ce qui fait échouer un test.

Où tu travailles : verification.py

Cas de test à écrire

L’entrée est ce que tu donnes au code ; l’attendu est ce que ton test doit vérifier.

T1 - Liste valide et ordre conservé

- Entrée : [1, 8000, 65535]
- Résultat attendu : La même liste, dans le même ordre, est acceptée.

Actions dans l’ordre

1. Crée verification.py à la racine du dépôt. Il importe portpilot et contient une fonction test_t1 qui vérifie que [1, 8000, 65535] est accepté, avec le même ordre et les mêmes valeurs.
2. Dans le test, une vérification s'écrit avec assert : si la condition est fausse, Python lève une AssertionError et le test échoue.
3. Définir test_t1 ne suffit pas : appelle-la explicitement à la fin de verification.py. Fais de même pour chaque test ajouté ensuite. Le message de succès vient après la vérification, jamais à sa place.
4. Lance verification.py. Il doit échouer, car portpilot.py n'existe pas encore : c'est la phase « rouge », elle est normale.
5. Ne commite rien à ce stade : le commit viendra quand le test passera.

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d'exemple.

```
def doubler(n):
    return n * 2

assert doubler(4) == 8, "doubler(4) doit valoir 8"
print("la condition est vraie, rien ne se passe")
```

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
python3 verification.py
```

Tu sais que c'est bon quand : python3 verification.py se termine par une erreur (ModuleNotFoundError ou AssertionError). Tu sais dire pourquoi.

Indice si tu bloques : Lis la dernière ligne du message d'erreur : elle indique le module manquant ou la valeur qui ne correspond pas. Pour l'exemple doubler, remplace 8 par 9 si tu veux voir un assert échouer, puis remets 8.

Palier 3 - T1 vert : le minimum de code (10 min)

Écrire le minimum de code qui fait passer T1, puis faire ton premier commit de code.

Où tu travailles : portpilot.py ; verification.py

Actions dans l'ordre

1. Crée portpilot.py avec la fonction valider_ports(ports). Pour l'instant, ton but est seulement de faire passer T1 : la fonction n'a pas à être complète.
2. Relance verification.py : T1 doit passer.
3. Quand T1 passe, fais ton premier commit. Le message est donné ci-dessous.
4. Vérifie avec git status que ni __pycache__ ni autre fichier inutile n'est ajouté. Ton .gitignore doit déjà exclure les caches Python.

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
python3 verification.py
```

Terminal · Relire l'état du dépôt

```
git status
```

Terminal · Enregistrer le travail vérifié

```
git add portpilot.py verification.py
git commit -m "Ajoute valider_ports et le test T1"
```

Terminal · Relire l'état du dépôt

```
git log --oneline
```

Tu sais que c'est bon quand : verification.py affiche T1 comme réussi, et git log --oneline montre ton commit « Ajoute valider_ports et le test T1 ».

Indice si tu bloques : Le minimum de code n'est pas la solution finale : les règles suivantes arriveront palier après palier, chacune avec son test. Un commit par palier de code garde l'historique lisible.

Palier 4 - Code de sortie et T2 : la liste vide (10 min)

Comprendre pourquoi un processus doit rendre un code non nul en cas d'échec, puis écrire T2.

Où tu travailles : portpilot.py ; verification.py

Pour comprendre

- Un programme se termine par un code de sortie : 0 pour une réussite, une autre valeur pour un échec. C'est ce code que lit le pipeline GitLab, pas le texte affiché.

Cas de test à écrire

L'entrée est ce que tu donnes au code ; l'attendu est ce que ton test doit vérifier.

T2 - Liste vide

- Entrée : []
- Résultat attendu : ValueError.

Actions dans l'ordre

1. Exemple sans rapport avec ton exercice : le programme ci-dessous rend 1 à cause de sys.exit(1). Lance-le puis lis le code avec echo $? (zsh).
2. Un test négatif (T2) attend une ValueError. Si aucune exception n'est levée, c'est un échec. Si une autre exception est levée, c'est aussi un échec : tu ne laisses pas passer une erreur différente.
3. Ajoute T2 dans verification.py : elle appelle valider_ports([]). Dans verification.py, appelle chaque test explicitement, compte les échecs, et termine avec sys.exit(1) s'il y en a au moins un.
4. Fais passer T2 en rejetant la liste vide, puis vérifie que T1 et T2 passent ensemble.
5. Vérifie qu'une assertion ratée rend bien un code non nul : tu peux temporairement modifier une valeur attendue dans un test, observer le code, puis remettre la valeur juste.

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d'exemple.

```
import sys

print("un test passe")
print("un autre échoue")
sys.exit(1)
```

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs. Lance le programme et echo $? ensemble : le nombre

affiché est le code du programme ; la console peut afficher code 0 pour echo lui-même.

```
python3 exemple.py; echo $?
```

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d'exemple.

```
def diviser(a, b):
    if b == 0:
        raise ValueError("diviseur nul")
    return a / b

try:
    diviser(1, 0)
except ValueError:
    print("attendu : ValueError levée")
```

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs. Lance le programme et echo $? ensemble : le nombre affiché est le code du programme ; la console peut afficher code 0 pour echo lui-même.

```
python3 verification.py; echo $?
```

Terminal · Enregistrer le travail vérifié

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git add portpilot.py verification.py
git commit -m "Rejette la liste vide dans valider_ports"
```

Tu sais que c'est bon quand : python3 verification.py; echo $? affiche 0 quand tout passe et une valeur non nulle quand une assertion est ratée. T2 passe.

Indice si tu bloques : Pour savoir si une exception a bien été levée, un bloc try avec except ValueError ne suffit pas : il faut aussi traiter le cas où rien n'est levé. Le mot-clé else d'un try peut t'aider. Ne mets pas python -O : il désactive les assert.

Palier 5 - T3 et T5 : onze ports, doublon (10 min)

Ajouter deux règles simples (limite de nombre et unicité), chacune avec son test.

Où tu travailles : portpilot.py ; verification.py

Cas de test à écrire

L'entrée est ce que tu donnes au code ; l'attendu est ce que ton test doit vérifier.

T3 - Onze ports distincts

- Entrée : Onze entiers distincts valides
- Résultat attendu : ValueError.

T5 - Doublon

- Entrée : [8000, 8000]
- Résultat attendu : ValueError.

Actions dans l'ordre

1. Ajoute T3 : onze entiers distincts et valides doivent lever ValueError. Ajoute T5 : [8000, 8000] doit lever ValueError.
2. Une liste a une longueur : len(liste) donne son nombre d'éléments. Un ensemble (set) ne garde pas les doublons.
3. Exemple sans rapport : une liste de fruits avec un doublon. Lance-le pour voir la différence de taille.
4. Complète valider_ports pour refuser plus de dix éléments et les doublons, sans casser T1 et T2.
5. Relance verification.py après chaque règle ajoutée, puis fais un commit.

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d'exemple.

```
fruits = ["pomme", "poire", "pomme"]
print(len(fruits))
print(len(set(fruits)))
```

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
python3 verification.py
```

Terminal · Enregistrer le travail vérifié

```
git add portpilot.py verification.py
git commit -m "Limite à dix ports et refuse les doublons"
```

Tu sais que c'est bon quand : T1, T2, T3 et T5 passent, et python3 verification.py se termine avec le code 0.

Indice si tu bloques : Compare la longueur de la liste à ce qui est autorisé avant de tester les doublons. Pour les doublons, compare le nombre d'éléments de la liste avec celui de la version en ensemble : c'est une idée, pas la fonction finale.

Palier 6 - T4 : six valeurs invalides, testées séparément (10 min)

Tester chaque valeur invalide dans sa propre liste, et comprendre pourquoi bool passe pour un entier.

Où tu travailles : portpilot.py ; verification.py

Cas de test à écrire

L'entrée est ce que tu donnes au code ; l'attendu est ce que ton test doit vérifier.

T4 - Valeurs invalides

- Entrée : -1, 0, 65536, texte, décimal et booléen, testés séparément
- Résultat attendu : ValueError pour chacun ; aucune autre exception ne passe.

Actions dans l'ordre

1. T4 couvre six cas testés séparément, chacun dans une liste d'un seul élément : [-1], [0], [65536], [« 8000 »] (une chaîne), [80.5] (un décimal) et [True] (un booléen).
2. Pour que l'échec d'un cas ne cache pas les autres, écris une boucle sur les six cas dans verification.py : chaque cas est vérifié séparément et son nom apparaît si la règle échoue.
3. Exemple sans rapport : en Python, True est un entier pour isinstance. Lance le code ci-dessous pour le constater.

4. Complète valider_ports pour que les six cas lèvent ValueError, en contrôlant les booléens avant les entiers. Relance verification.py et fais un commit.

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d'exemple.

```
print(isinstance(3, int))
print(isinstance(True, int))
print(isinstance("3", int))
```

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
python3 verification.py
```

Terminal · Enregistrer le travail vérifié

```
git add portpilot.py verification.py
git commit -m "Refuse les valeurs invalides, booléens compris"
```

Tu sais que c'est bon quand : Les six sous-cas de T4 passent séparément ; le nom d'un cas en échec serait affiché.

Indice si tu bloques : Un booléen est une sorte d'entier pour Python. Refuse d'abord True et False avant d'accepter un entier. Pour le décimal, écris bien 80.5 en Python : 80,5 seul serait une liste de deux entiers.

Palier 7 - CI : fichier, MR et premier pipeline vert (10 min)

Relier la même commande à GitLab : un job qui n'exécute que sur les merge requests, puis une première MR verte.

Où tu travailles : .gitlab-ci.yml ; MR GitLab

Pour comprendre

- Documentation GitLab : https://docs.gitlab.com/ci/pipelines/merge_request_pipelines/

Actions dans l'ordre

1. Le fichier .gitlab-ci.yml est à la racine du dépôt. Tu le rédiges toi-même ; ce parcours ne te fournit pas sa version complète.
2. Définis la politique de pipeline pour merge_request_event uniquement : les pipelines de MR se lancent, les simples push sur une branche sans MR ne se lancent pas.
3. Ajoute un job nommé verifier dont la commande est exactement celle que tu lances en local : python verification.py (ou python3 selon l'environnement).
4. N'ajoute ni allow_failure, ni job manuel, ni règle finale permissive qui lancerait tout. Le job doit être bloquant.
5. Le job Lint éventuel contrôle la syntaxe du fichier, pas le fonctionnement du runner : un job Lint vert ne prouve pas que ton job verifier tourne.
6. Commite le fichier, pousse la branche, ouvre la MR non Draft vers la branche principale, puis vérifie le pipeline du dernier commit.

Terminal · Enregistrer le travail vérifié

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git add .gitlab-ci.yml
git commit -m "Ajoute la CI de merge request avec le job verifier"
```

Terminal · Publier la branche ou ouvrir la demande de fusion

```
git push -u origin feat/validation
```

Terminal · Relire l'état du dépôt

```
git log --oneline -1
```

Tu sais que c'est bon quand : La MR existe, son pipeline tourne sur le dernier commit et le job verifier est vert. Ce vert est un résultat de pipeline, pas un test local.

Indice si tu bloques : Si aucun pipeline ne se lance, vérifie la règle merge_request_event et que la MR est bien ouverte depuis feat/validation. Un test local réussi ne suffit pas : ce qui compte pour la fusion est le pipeline du dernier commit de la MR.

Palier 8 - Régression, rouge bloquant, correction et fusion (10 min)

Observer qu'un pipeline rouge bloque la fusion, corriger le code sans toucher aux tests, puis fusionner le dernier commit vert.

Où tu travailles : portpilot.py ; MR GitLab

Actions dans l'ordre

1. Introduis une petite régression réelle dans portpilot.py, par exemple en retirant l'une des règles déjà écrites. Ne modifie ni verification.py ni les tests.
2. Commite, pousse, puis attends le pipeline. Il doit passer au rouge. Note le lien du pipeline rouge et le motif du blocage : le test en échec et le message.
3. Observe le blocage de la fusion dans la MR : le bouton ne doit pas être utilisable tant que le pipeline n'est pas réussi (règle de fusion de l'école).
4. Corrige le code, commite la correction, pousse, puis vérifie que le pipeline du dernier commit est vert.
5. Relis le diff de la MR, fusionne le dernier commit vert, puis reviens sur la branche principale : git switch main, git pull --ff-only.
6. Garde le lien du pipeline rouge : il sert de preuve pour la suite du parcours.

Terminal · Relire l'état du dépôt

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git diff
```

Terminal · Enregistrer le travail vérifié

```
git add portpilot.py
git commit -m "Introduit une régression volontaire pour tester le blocage"
```

Terminal · Publier la branche ou ouvrir la demande de fusion

```
git push
```

Terminal · Enregistrer le travail vérifié

```
git add portpilot.py
git commit -m "Corrige la régression de valider_ports"
```

Terminal · Publier la branche ou ouvrir la demande de fusion

```
git push
```

Terminal · Après la fusion : actualiser la principale

```
git switch main
```

Terminal · Commande à lancer au moment indiqué

```
git pull --ff-only
```

Tu sais que c’est bon quand : Tu as un pipeline rouge avec son lien et son motif, puis un pipeline vert sur le dernier commit, et la MR est fusionnée. Un rouge bloque la fusion ; un vert seul ne prouve pas qu’elle est autorisée.

Indice si tu bloques : La régression doit venir du code, jamais des tests : si tu modifies un test pour le faire passer, tu affaiblis la vérification. Le message de blocage et la règle de fusion sont à lire dans l’interface GitLab de la MR.

Avant de livrer

- Pour T2 à T5, ValueError doit être levée ; son absence est un échec, et toute autre exception est également un échec. Les six sous-cas T4 sont testés indépendamment.
- Le pipeline de MR teste le dernier commit, un compte habilité constate le blocage lors du rouge, puis la correction donne un dernier pipeline vert.
- Trois fonctions de contrôle GitLab sont vérifiées : MR ouverte, pipeline de MR, règle qui exige un pipeline réussi.

MR2 - feat/connexion (70 min)

Cette fonction sera appelée pour chaque port en MR3. Avance dans l’ordre des paliers et vérifie chaque résultat avant de poursuivre.

Avant de commencer

- Point de départ : MR1 fusionnée ; les cas T1 à T5 passent et le pipeline existe.
- Ce que tu vas produire : tester_port dans portpilot.py et le cas T6 dans verification.py : une connexion locale, quatre états possibles, socket fermée.
- Pourquoi : Cette fonction sera appelée pour chaque port en MR3.
- Commande des tests depuis la racine : `python3 verification.py`. Les cas ci-dessous sont à écrire par toi.

Palier 1 - Repartir de main à jour (10 min)

Créer la branche de la MR 2 depuis la branche principale qui contient déjà la MR 1 fusionnée.

Où tu travailles : Dépôt

Actions dans l’ordre

1. Vérifie que la MR 1 est bien fusionnée dans la branche principale. Si ce n’est pas le cas, arrête-toi et règle ça d’abord.
2. Reviens sur la branche principale, récupère ses derniers commits, puis crée feat/connexion.
3. Lance verification.py avant de commencer : T1 à T5 doivent passer sur cette base.

Terminal · Relire l’état du dépôt

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git status
```

Terminal · Après la fusion : actualiser la principale

```
git switch main
```

Terminal · Commande à lancer au moment indiqué

```
git pull --ff-only
```

Terminal · Commande à lancer au moment indiqué

```
git switch -c feat/connexion
```

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs. Lance le programme et echo $? ensemble : le nombre affiché est le code du programme ; la console peut afficher code 0 pour echo lui-même.

```
python3 verification.py; echo $?
```

Tu sais que c'est bon quand : git status affiche « On branch feat/connexion », le dépôt est propre, et verification.py se termine avec le code 0.

Indice si tu bloques : Si ta branche principale n'a pas la fusion de la MR 1, git pull --ff-only ne peut pas le deviner : vérifie le graphe avec git log --oneline --graph.

Palier 2 - Socket d'écoute et port 0 : T6 rouge (10 min)

Comprendre une socket d'écoute locale, puis écrire T6 qui en ouvre une et appelle tester_port.

Où tu travailles : verification.py

Cas de test à écrire

L'entrée est ce que tu donnes au code ; l'attendu est ce que ton test doit vérifier.

T6 - Connexion à une écoute éphémère

- Entrée : Le port attribué par le système à une écoute TCP locale encore ouverte
- Résultat attendu : tester_port retourne exactement OUVERT.

Actions dans l'ordre

1. Une socket est un point de communication réseau. Une socket d'écoute attend des connexions : tu la lies à une adresse (bind), tu la mets en écoute (listen), et tu lis le port que le système a choisi (getsockname).
2. Le port 0 demande au système un port libre. Ce port n'est jamais une cible de scan : il sert uniquement à créer l'écoute du test.
3. Exemple sans rapport avec ton exercice : le programme ci-dessous ouvre une écoute et affiche le port attribué. Lance-le pour voir le résultat.
4. Dans verification.py, écris test_t6 : ouvre une socket IPv4 liée à 127.0.0.1 et au port 0, appelle listen, garde l'écoute ouverte pendant l'appel à tester_port et vérifie que le résultat est exactement OUVERT.
5. Lance verification.py : il doit échouer, car tester_port n'existe pas encore. C'est la phase rouge.

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d'exemple.

```
import socket

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as ecoute:
    ecoute.bind(("127.0.0.1", 0))
    ecoute.listen()
    print("port attribué :", ecoute.getsockname()[1])
```

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
python3 verification.py
```

Tu sais que c'est bon quand : verification.py échoue, et le message montre que tester_port n'est pas encore disponible.

Indice si tu bloques : Le getsockname() d'une socket liée au port 0 donne un couple (adresse, port) : le port est le second élément. Pas de accept, pas de thread, pas de time.sleep dans ce test.

Palier 3 - tester_port minimal : T6 vert (10 min)

Écrire tester_port avec l'hôte constant 127.0.0.1, le délai de 0,5 seconde et une seule tentative, puis faire passer T6.

Où tu travailles : portpilot.py ; verification.py

Actions dans l'ordre

1. Ajoute tester_port(port) dans portpilot.py. L'hôte est la constante 127.0.0.1, le délai est de 0,5 seconde, une seule tentative.
2. socket.create_connection((hote, port), timeout=...) tente la connexion. Lis son aide pour voir ses arguments : python3 -c "import socket; help(socket.create_connection)" (appuie sur q pour sortir).
3. Le minimum pour T6 : une connexion réussie retourne OUVERT. Tu traiteras les autres cas au palier suivant.
4. Relance verification.py : T1 à T6 doivent passer.
5. Fais un commit pour T6 et tester_port.

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
python3 -c "import socket; help(socket.create_connection)"
```

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs. Lance le programme et echo $? ensemble : le nombre affiché est le code du programme ; la console peut afficher code 0 pour echo lui-même.

```
python3 verification.py; echo $?
```

Terminal · Enregistrer le travail vérifié

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git add portpilot.py verification.py
git commit -m "Ajoute tester_port et le test T6"
```

Tu sais que c'est bon quand : T1 à T6 passent et git log --oneline montre ton commit « Ajoute tester_port et le test T6 ».

Indice si tu bloques : Une connexion qui réussit ne signifie que l'ouverture TCP. Ne lis pas d'autre donnée sur la socket : aucune donnée applicative n'est envoyée par PortPilot.

Palier 4 - Les autres états : REFUSE, TIMEOUT, ERREUR (10 min)

Classer les exceptions réseau dans le bon état, avec le bon ordre des except.

Où tu travailles : portpilot.py

Actions dans l'ordre

1. tester_port retourne exactement OUVERT, REFUSE, TIMEOUT ou ERREUR. REFUSE correspond à ConnectionRefusedError, TIMEOUT à un dépassement du délai, ERREUR à tout autre OSError.
2. Les exceptions sont classées en arbre : ConnectionRefusedError est une sorte d'OSError. Un except OSError placé en premier attrape donc tout, y compris les cas plus précis.
3. Exemple sans rapport avec ton exercice : ouvrir un fichier qui n'existe pas. Lance-le pour voir quelle exception apparaît, puis compare l'ordre des except.
4. Ajoute les branches REFUSE, TIMEOUT et ERREUR dans tester_port, en mettant les exceptions précises avant OSError. T6 doit toujours passer.
5. Relis le code et vérifie l'ordre : une exception précise doit toujours être traitée avant son parent.

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d'exemple.

```
try:
    with open("fichier_inexistant.txt") as f:
        print(f.read())
except FileNotFoundError:
    print("fichier absent : exception précise")
except OSError:
    print("autre erreur du système")
```

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs. Lance le programme et echo $? ensemble : le nombre affiché est le code du programme ; la console peut afficher code 0 pour echo lui-même.

```
python3 verification.py; echo $?
```

Terminal · Enregistrer le travail vérifié

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git add portpilot.py
git commit -m "Classe les erreurs de connexion en REFUSE, TIMEOUT et ERREUR"
```

Tu sais que c'est bon quand : tester_port peut retourner les quatre états, dans cet ordre de traitement, et T1 à T6 passent toujours.

Indice si tu bloques : Compte les exceptions de la plus précise à la plus générale. Pour TIMEOUT, vérifie dans l'aide de Python quelle exception correspond au dépassement du délai dans ta version de Python.

Palier 5 - Fermer la socket, même en cas d'erreur (10 min)

Garantir la fermeture de toute socket créée, même quand une exception survient.

Où tu travailles : portpilot.py

Actions dans l'ordre

1. Une socket ouverte et non fermée reste allouée. Avec with, Python ferme l'objet à la sortie du bloc, même en cas d'exception.
2. Exemple sans rapport : finally s'exécute toujours, qu'une exception soit levée ou non. Lance le code ci-dessous avec un diviseur nul puis avec un diviseur valide.

3. Relis tester_port : toute socket créée doit être fermée, y compris quand une branche d’erreur est prise.
4. Relance verification.py. Tu ne modifies pas T6 : seul le code de tester_port peut changer ici.

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d’exemple.

```
def diviser(a, b):
    try:
        return a / b
    finally:
        print("nettoyage fait")

print(diviser(6, 3))
diviser(1, 0)
```

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs. Lance le programme et echo $? ensemble : le nombre affiché est le code du programme ; la console peut afficher code 0 pour echo lui-même.

```
python3 verification.py; echo $?
```

Tu sais que c’est bon quand : Dans tester_port, chaque socket créée est fermée dans tous les cas, et verification.py passe toujours.

Indice si tu bloques : Un bloc with n’a pas besoin de close() explicite. Si tu crées une socket en dehors d’un with, pense à finally pour la fermer.

Palier 6 - Relecture, commit et pipeline de MR (10 min)

Relire ton diff, pousser la branche et vérifier que la CI de MR teste bien le dernier commit.

Où tu travailles : Branche feat/connexion ; MR GitLab

Actions dans l’ordre

1. Lis git status et git diff avant tout ajout : tu ne veux ni __pycache__, ni fichier inutile.
2. Vérifie que le test n’emploie aucun port fixe, aucun serveur externe, aucun thread, aucun accept et aucune attente artificielle.
3. Ajoute explicitement les fichiers nécessaires, fais un commit descriptif, pousse la branche et ouvre la MR vers la branche principale.
4. Vérifie que le pipeline du dernier commit de la MR exécute toute la suite : T1 à T6. Ce pipeline ne dépend ni du labo ni d’un service déjà présent.

Terminal · Relire l’état du dépôt

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git status
git diff
```

Terminal · Enregistrer le travail vérifié

```
git add portpilot.py verification.py
git commit -m "Termine la connexion locale et relit les états d’erreur"
```

Terminal · Publier la branche ou ouvrir la demande de fusion

```
git push -u origin feat/connexion
```

Terminal · Relire l’état du dépôt

```
git log --oneline -3
```

Tu sais que c’est bon quand : La MR 2 existe, son pipeline sur le dernier commit est vert et il exécute T1 à T6.

Indice si tu bloques : Un pipeline vert sur un commit précédent ne suffit pas : c’est le dernier commit de la MR qui doit être testé. Si la CI ne lance pas T6, regarde la commande de ton job.

Palier 7 - Fusion et retour sur main (10 min)

Fusionner la MR 2 après un pipeline vert, puis repartir d’une branche principale à jour pour la MR 3.

Où tu travailles : MR GitLab ; branche principale

Actions dans l’ordre

1. Relis le diff de la MR dans GitLab, vérifie que le pipeline du dernier commit est vert, puis fusionne.
2. Une fusion autorisée n’est pas la même chose qu’un test local réussi : ce qui compte pour l’école est le pipeline du dernier commit et la règle de fusion.
3. Reviens sur la branche principale et mets-la à jour. Ta branche de la MR 3 se crée à partir de cette version.

Terminal · Après la fusion : actualiser la principale

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git switch main
```

Terminal · Commande à lancer au moment indiqué

```
git pull --ff-only
```

Terminal · Relire l’état du dépôt

```
git log --oneline -5
```

Tu sais que c’est bon quand : La MR 2 est fusionnée après pipeline vert du dernier commit, et git log --oneline sur main montre le commit de fusion.

Indice si tu bloques : Si GitLab refuse la fusion, lis le motif affiché dans la MR : il s’agit souvent d’un pipeline pas encore réussi ou d’une règle de fusion.

Avant de livrer

- T1 à T6 restent appelés et passent localement et dans un runner ordinaire.
- Le test n’emploie aucun port fixe, serveur externe, thread, accept ou attente artificielle.
- La socket d’écoute est fermée dans tous les cas.

MR3 - feat/scan-bilan (80 min)

Tu assembles validation, connexion et bilan sans élargir le périmètre du scanner. Avance dans l’ordre des paliers et vérifie chaque résultat avant de poursuivre.

Avant de commencer

- Point de départ : MR2 fusionnée ; T1 à T6 passent sur la branche principale.
- Ce que tu vas produire : scanner_ports, resumer et main ; T7 et T8 ; README et livraison ZIP par la CI.
- Pourquoi : Tu assembles validation, connexion et bilan sans élargir le périmètre du scanner.
- Commande des tests depuis la racine : `python3 verification.py`. Les cas ci-dessous sont à écrire par toi.

Palier 1 - Branche à jour et liste des livrables (10 min)

Partir de la branche principale à jour et savoir exactement ce que la MR 3 doit livrer.

Où tu travailles : Dépôt

Actions dans l'ordre

1. Vérifie que la MR 2 est fusionnée, puis reviens sur la branche principale et mets-la à jour.
2. Crée feat/scan-bilan. Relis la liste des livrables : portpilot.py, verification.py, README.md, .gitignore et .gitlab-ci.yml. Le ZIP ne contient que portpilot.py, verification.py et README.md.
3. Lance verification.py : T1 à T6 doivent passer sur cette base avant tout ajout.

Terminal · Relire l'état du dépôt

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git status
```

Terminal · Après la fusion : actualiser la principale

```
git switch main
```

Terminal · Commande à lancer au moment indiqué

```
git pull --ff-only
```

Terminal · Commande à lancer au moment indiqué

```
git switch -c feat/scan-bilan
```

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs. Lance le programme et echo $? ensemble : le nombre affiché est le code du programme ; la console peut afficher code 0 pour echo lui-même.

```
python3 verification.py; echo $?
```

Tu sais que c'est bon quand : git status affiche « On branch feat/scan-bilan », le dépôt est propre, et verification.py se termine avec le code 0.

Indice si tu bloques : Si T6 échoue sur la branche principale, ne continue pas : la MR 2 n'est pas stable. Compare avec git log --oneline.

Palier 2 - Valider toute la liste d'abord : T7 rouge (10 min)

Comprendre pourquoi on valide l'ensemble avant d'agir, puis écrire T7 qui doit échouer.

Où tu travailles : verification.py

Cas de test à écrire

L'entrée est ce que tu donnes au code ; l'attendu est ce que ton test doit vérifier.

T7 - Scan de deux écoutes dans l'ordre

- Entrée : Deux sockets éphémères ouvertes, fournies dans un ordre choisi
- Résultat attendu : Deux paires port/OUVERT exactement dans l'ordre de l'entrée.

Actions dans l'ordre

1. Quand une action se fait pour plusieurs éléments, il vaut mieux vérifier toute la liste avant la première action. Ainsi une entrée invalide ne provoque aucune connexion partielle.

2. Exemple sans rapport avec ton exercice : préparer un panier de courses. Le programme ci-dessous vérifie toute la liste avant d'ajouter le premier article. Lance-le avec la liste valide puis avec une liste contenant une valeur invalide.
3. Dans verification.py, écris test_t7 : deux écoutes éphémères distinctes, toutes deux ouvertes pendant le scan, puis vérification que scanner_ports retourne exactement les deux paires [port, OUVERT] dans l'ordre donné.
4. Ferme les deux sockets systématiquement, même si une assertion échoue.
5. Lance verification.py : T7 doit échouer, car scanner_ports n'existe pas encore.

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d'exemple.

```
articles = ["pain", "", "lait"]

for article in articles:
    if article == "":
        print("liste refusée : article vide")
        break
else:
    print("liste valide, on commence les achats")
```

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
python3 verification.py
```

Tu sais que c'est bon quand : T7 est écrit, il échoue, et tu peux dire pourquoi.

Indice si tu bloques : Le test doit garder les deux écoutes ouvertes tant que scanner_ports n'a pas fini. Un with imbriqué ou un try/finally peut garantir la fermeture des deux sockets.

Palier 3 - scanner_ports : T7 vert et ordre conservé (10 min)

Écrire scanner_ports qui valide la liste entière, fait une tentative par port dans l'ordre reçu, puis faire passer T7.

Où tu travailles : portpilot.py ; verification.py

Actions dans l'ordre

1. scanner_ports(ports) valide toute la liste avec valider_ports avant la première connexion. Une liste invalide ne doit donc déclencher aucune tentative.
2. Pour chaque port de la liste, dans l'ordre, fais une seule tentative avec tester_port et ajoute la paire [port, état] à la liste de résultats.
3. Exemple sans rapport : une boucle for qui construit une liste de paires avec leur indice. Lance-le pour voir l'ordre conservé.
4. Relance verification.py jusqu'à ce que T1 à T7 passent, puis fais un commit.

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d'exemple.

```
fruits = ["pomme", "poire"]
resultats = []
```

```
for position, fruit in enumerate(fruits):
    resultats.append([position, fruit])
print(resultats)
```

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs. Lance le programme et echo $? ensemble : le nombre affiché est le code du programme ; la console peut afficher code 0 pour echo lui-même.

```
python3 verification.py; echo $?
```

Terminal · Enregistrer le travail vérifié

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git add portpilot.py verification.py
git commit -m "Ajoute scanner_ports et le test T7"
```

Tu sais que c'est bon quand : T1 à T7 passent et git log --oneline montre ton commit « Ajoute scanner_ports et le test T7 ».

Indice si tu bloques : scanner_ports ne doit pas réimplémenter la connexion : il réutilise tester_port. Un port invalide dans la liste doit faire échouer la validation avant la boucle de connexion.

Palier 4 - resumer et T8 : le bilan (10 min)

Compter les états dans un dictionnaire à cinq compteurs, et écrire T8 pour vérifier le comptage.

Où tu travailles : portpilot.py ; verification.py

Cas de test à écrire

L'entrée est ce que tu donnes au code ; l'attendu est ce que ton test doit vérifier.

T8 - Bilan mixte et vide

- Entrée : Deux OUVERT, un REFUSE, un TIMEOUT, un ERREUR ; puis liste vide
- Résultat attendu : total 5, ouverts 2, refuses 1, timeouts 1, erreurs 1 ; cinq zéros pour la liste vide.

Actions dans l'ordre

1. resumer(resultats) retourne cinq compteurs entiers : total, ouverts, refuses, timeouts et erreurs. Les catégories doivent totaliser total. Une liste vide donne cinq zéros.
2. Exemple sans rapport : compter des fruits dans un dictionnaire. Lance le code et vérifie que chaque clé reçoit un compteur.
3. Dans verification.py, écris test_t8 : il construit deux OUVERT, un REFUSE, un TIMEOUT et un ERREUR, vérifie les cinq compteurs, puis vérifie les cinq zéros pour une liste vide.
4. T8 vérifie le comptage d'états fabriqués, il ne provoque pas de vraies erreurs réseau.
5. Écris resumer, relance verification.py, puis fais un commit.

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d'exemple.

```
fruits = ["pomme", "poire", "pomme"]
compte = {}
for fruit in fruits:
    compte[fruit] = compte.get(fruit, 0) + 1
print(compte)
```

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs. Lance le programme et echo $? ensemble : le nombre affiché est le code du programme ; la console peut afficher code 0 pour echo lui-même.

```
python3 verification.py; echo $?
```

Terminal · Enregistrer le travail vérifié

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git add portpilot.py verification.py
git commit -m "Ajoute resumer et le test T8"
```

Tu sais que c'est bon quand : T1 à T8 passent, et tu peux expliquer pourquoi T8 ne prouve pas que les erreurs réseau se déclenchent.

Indice si tu bloques : Commence par le cas de la liste vide : un dictionnaire ou des compteurs à zéro suffisent. Vérifie que la somme des catégories égale total dans ton test.

Palier 5 - main : message clair et import sans lancement (10 min)

Séparer le point d'entrée des fonctions, pour que l'import du module ne lance jamais le scan.

Où tu travailles : portpilot.py

Actions dans l'ordre

1. Le bloc if __name__ == "__main__": ne s'exécute que lorsque tu lances le fichier directement. Quand verification.py importe portpilot, ce bloc ne doit donc pas tourner.
2. Exemple sans rapport : un module qui affiche quelque chose seulement s'il est lancé directement. Lance-le directement, puis importe-le depuis un autre fichier.
3. Dans un exemple.py jetable, lance python3 exemple.py puis python3 -c "import exemple" : le premier affiche Bonjour, le second ne doit rien afficher. Supprime exemple.py après l'essai.
4. main coordonne la liste configurée, le scan, un résultat par port et le bilan. Une liste invalide affiche un message clair au lieu d'une trace d'erreur brute.
5. Relance verification.py : il doit toujours passer, et l'import ne doit pas afficher le scan.
6. Fais un commit.

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d'exemple.

```
def saluer():
    print("Bonjour")

if __name__ == "__main__":
    saluer()
```

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
python3 -c "import portpilot"
```

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs. Lance le programme et echo $? ensemble : le nombre affiché est le code du programme ; la console peut afficher code 0 pour echo lui-même.

```
python3 verification.py; echo $?
```

Terminal · Enregistrer le travail vérifié

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git add portpilot.py verification.py
git commit -m "Ajoute main et le message de liste invalide"
```

Tu sais que c'est bon quand : L'import de portpilot n'affiche rien, un lancement direct présente le bilan ou un message clair, et verification.py passe.

Indice si tu bloques : Pour le message clair, attrape ValueError autour de la validation de la liste dans main et affiche le texte de l'erreur. Le fichier principal ne doit pas planter avec une trace inutile pour une liste invalide.

Palier 6 - README final avant la CI (10 min)

Décrire le projet et ses limites avant que la CI de livraison ne construise l'artefact.

Où tu travailles : README.md

Actions dans l'ordre

1. Le README décrit l'objectif et ses limites, les prérequis, la configuration de la liste, le lancement, les quatre états, les tests et le téléchargement de l'artefact.
2. Le scan ne vise que 127.0.0.1, de façon séquentielle, avec une tentative et 0,5 seconde par port. Un état OUVERT ne prouve ni vulnérabilité ni nature du service.
3. Précise que T8 compte des états fabriqués : les états REFUSE, TIMEOUT et ERREUR ne sont pas déclenchés par la suite de tests.
4. Pour la démonstration locale, prévois le port 8000 qui est celui du labo de Cyber Atelier : tu l'écriras dans la liste configurée au palier 8.
5. Fais un commit du README.

Terminal · Relire l'état du dépôt

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git status
```

Terminal · Enregistrer le travail vérifié

```
git add README.md
git commit -m "Finalise le README de PortPilot"
```

Tu sais que c'est bon quand : Le README contient les sections demandées, il est cohérent avec ce que fait le code et il est commité.

Indice si tu bloques : Un README qui promet trop est une erreur : relis les limites dans le guide du parcours et écris-les avec tes mots. Un test local ou un pipeline vert ne prouve pas la sécurité du scan.

Palier 7 - CI en deux stages et artefact ZIP (10 min)

Ajouter une étape de livraison qui ne tourne qu'après les tests et produit un ZIP contrôlé.

Où tu travailles : .gitlab-ci.yml ; portpilot.zip

Actions dans l'ordre

1. Ton .gitlab-ci.yml a deux stages successifs : test, puis livraison. Le job de livraison ne s'exécute qu'après le succès du stage test.
2. Le job de livraison crée portpilot.zip avec exactement portpilot.py, verification.py et README.md, et le conserve comme artefact téléchargeable pendant sept jours.
3. Exemple sans rapport avec ton exercice : zipfile crée une archive avec des fichiers choisis. Lance-le et liste le contenu avec python3 -m zipfile -l.
4. Vérifie localement ton ZIP sur les trois fichiers attendus avant de pousser. Le pipeline reste la référence : il doit produire le même contenu.
5. Commite la CI, pousse, puis vérifie le pipeline du dernier commit : le stage test doit être vert avant la livraison.

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d'exemple.

```
import zipfile

with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("exemple\n")
with zipfile.ZipFile("exemple.zip", "w") as archive:
    archive.write("notes.txt")
```

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
python3 -m zipfile -l exemple.zip
```

Terminal · Enregistrer le travail vérifié

```
git add .gitlab-ci.yml
git commit -m "Ajoute les stages test et livraison de PortPilot"
```

Terminal · Publier la branche ou ouvrir la demande de fusion

```
git push
```

Terminal · Relire l'état du dépôt

```
git log --oneline -1
```

Tu sais que c'est bon quand : Le pipeline du dernier commit affiche le stage test vert, puis la livraison, et l'artefact apparaît dans la page du pipeline.

Indice si tu bloques : Les règles de livraison ne doivent pas contourner le stage test : la livraison suit les tests. Supprime les fichiers jetables de l'exemple (notes.txt, exemple.zip) avant de commiter.

Palier 8 - Démonstration locale, artefact et fusion (10 min)

Constater le comportement du scan sur le labo, vérifier l'artefact du dernier pipeline vert, puis fusionner la MR 3.

Où tu travailles : MR GitLab

Actions dans l'ordre

1. Démonstration hors tests : dans le dossier de Cyber Atelier, lance python3 labs.py dans un premier terminal. Le labo répond sur 127.0.0.1:8000.

2. Dans un second terminal, dans workspaces/portpilot, lance python3 portpilot.py avec 8000 dans la liste configurée. Attends OUVERT pour ce port.
3. Arrête labs.py avec Ctrl+C, relance portpilot.py : tu dois voir REFUSE. Ne lance aucun autre serveur sur 8000, le labo suffit.
4. Cette démonstration n'est pas un test : elle dépend du labo, et verification.py ne doit jamais dépendre de lui.
5. Vérifie l'artefact portpilot.zip du dernier pipeline vert avant son expiration, et ouvre-le pour contrôler ses trois fichiers.
6. Relis le diff de la MR, fusionne après pipeline vert du dernier commit, puis mets la branche principale à jour.

Terminal · Option Mac : démarrer le laboratoire dans un autre terminal

Terminal séparé sur ton Mac, depuis le dossier cyberatelier. Laisse-le ouvert pendant la démonstration, puis arrête avec Ctrl+C. Dans Docker, utilise plutôt le bouton du laboratoire.

```
python3 labs.py
```

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
python3 portpilot.py
```

Terminal · Commande à lancer au moment indiqué

```
python3 -m zipfile -l portpilot.zip
```

Terminal · Après la fusion : actualiser la principale

```
git switch main
```

Terminal · Commande à lancer au moment indiqué

```
git pull --ff-only
```

Terminal · Relire l'état du dépôt

```
git log --oneline -5
```

Tu sais que c'est bon quand : OUVERT pour 8000 pendant labs.py, REFUSE après Ctrl+C, l'artefact contient exactement trois fichiers, et la MR 3 est fusionnée après pipeline vert.

Indice si tu bloques : Le succès du script local ne décide pas si GitLab autorise une fusion ; vérifie aussi le commit et la règle de fusion. Garde le lien du pipeline rouge de la MR 1 comme preuve.

Avant de livrer

- L'import du module n'exécute pas main ; une liste invalide affiche un message clair.
- Les huit tests sont appelés ; T4 vérifie ses six sous-cas séparément. Les tests ne prétendent pas couvrir le déclenchement des trois états d'erreur réseau.
- portpilot.zip contient exactement portpilot.py, verification.py et README.md, généré après succès des tests et conservé sept jours.
- Trois MR sont fusionnées au pipeline vert du dernier commit ; la preuve du rouge MR1 est conservée.

Cycle de travail et validation

Pour chaque MR, développe et teste localement, lis git status et git diff, ajoute explicitement les fichiers nécessaires, crée des commits descriptifs et pousse sur la branche de la MR. Ouvre la MR vers la branche principale, vérifie le pipeline du dernier commit et relis le diff. Après chaque fusion, actualise la branche principale avant de créer la suivante. Une seule MR finale ne remplace pas les trois étapes.

Commande locale depuis la racine : python verification.py. Adapte python en python3 si nécessaire. Une assertion échouée doit rester visible et rendre le processus non nul. Ne lance pas Python en mode optimisé. Les huit tests couvrent validation valide, liste vide, trop d'éléments, six cas invalides testés séparément dans des listes à un élément, doublons, connexion ouverte, scan de deux ports dans l'ordre et bilan mixte/vide.

Livrables et limites

Le dépôt final contient portpilot.py, verification.py, README.md, .gitignore et .gitlab-ci.yml, trois MR fusionnées au dernier pipeline vert, le lien du pipeline rouge de MR1 avec son motif, et l'artefact portpilot.zip du pipeline vert de MR3. Le ZIP contient uniquement les trois fichiers annoncés.

L'outil tente seulement une connexion TCP sur 127.0.0.1, de façon séquentielle, avec une tentative et un délai de 0,5 seconde par port. Il ne prouve ni vulnérabilité, ni disponibilité générale, ni identité d'un service. Le projet ne couvre pas toutes les branches d'erreur réseau par tests automatisés. L'exercice concerne exclusivement un laboratoire local autorisé.